

Bitácora de Desarrollo: OPC-UA Industrial Explorer

Bitácora de Desarrollo: OPC-UA Industrial Explorer

Sistema de Comunicación Cliente-Servidor Industrial

1. Introducción y Requerimientos

El proyecto consiste en la creación de un ecosistema OPC-UA completo con servidor Python, cliente Python y panel HMI web. El servidor debe exponer variables industriales estándar y ser accesible por cualquier cliente OPC-UA del mercado (UaExpert, Prosys, SCADA, etc.).

2. Registro Detallado de Prompts y Funcionalidades con su análisis técnico

A continuación se presenta el registro cronológico completo de las interacciones, detallando el prompt original y su impacto técnico en el sistema.

Interacción #1

Prompt:

"El OPC-UA es muy complicado de usar, busca una solución para que sea más sencillo de usar."

Análisis Técnico:

Se creó un script unificado (unified_opcua.py) que combina servidor y cliente en un solo comando, eliminando la necesidad de configurar puertos y procesos separados.

Interacción #2

Prompt:

"El OPC-UA no está compuesto por servidor y cliente y debería, arrégalo."

****Análisis Technical:****

Se mantuvo la arquitectura separada:

- server_opcua.py: Servidor OPC-UA

- client_opcua.py: Cliente OPC-UA
- opcua_system.py: Script unificado con 3 modos (server, client, demo)

Interacción #3

Prompt:

"Del OPC-UA tengo que poder detener simulación, ver los logs, que el texto se lea bien."

Análisis Técnico:

Se implementaron controles en el HMI:

- Botones de INICIAR/DETENER simulación
- Terminales con logs de alta legibilidad
- Sistema de logs inteligentes con colores y timestamps
- Gráficos de tendencia dinámicos

Interacción #4

Prompt:

"El OPC-UA que funcione con servidor y cliente. Prepara el OPC-UA para que se pueda conectar a cualquier tipo de cliente."

Análisis Técnico:

Se configuró el servidor para máxima compatibilidad:

- SecurityPolicy: None (máxima compatibilidad)
- Endpoint: opc.tcp://0.0.0.0:4841/freeopcua/server/
- Namespace personalizado: http://izan.urios.industrial
- Variables escribibles por cualquier cliente

Corrección: Este prompt se usó para corregir la arquitectura, asegurando que cualquier cliente OPC-UA pueda conectarse (UaExpert, Prosys, Node-RED, SCADA).

Interacción #5

Prompt:

"El ejercicio 2.2 no es capaz de generar código, solo es capaz de validarlo. El SCL no es capaz de generar programas libres."

Análisis Técnico:

NOTA: Esta corrección pertenece al Ejercicio 2.3 (SCL), pero se documenta para referencia cruzada.

Interacción #6

Prompt:

"El SCL está perfecto. Mejora el OPC-UA. Remember that it must have a server and a client."

Análisis Técnico:

Se rediseñó la arquitectura completa:

- Servidor estándar (server_opcua.py) con 7 nodos
- Cliente separado (client_opcua.py) para monitorización
- HMI (scada_visual.html) con diagrama de arquitectura
- Dos terminales separadas: servidor y cliente
- Panel de clientes compatibles listados

Corrección: Este prompt se usó para corregir y mejorar el OPC-UA mientras se mantenía el SCL intacto.

3. Resumen de Funcionalidades Implementadas

|-----|-----|

4. Variables del Servidor OPC-UA

|-----|-----|-----|

5. Conclusión

El sistema OPC-UA se consolidó como un ecosistema profesional de comunicación industrial, con arquitectura servidor/cliente real, compatibilidad universal con cualquier cliente OPC-UA, y una interfaz HMI visual que facilita la monitorización y el control.

Documento realizado por Izan Urios de 3R de Automatización y Robótica Industrial

Documento realizado por Izan Urios de 3R de Automatización y Robótica Industrial